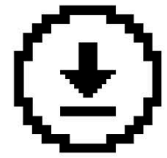


6 Create the circles

To draw a circle, use the Canvas widget's `create_oval()` function. Type the following code below the shapes list. It creates four circles of the same size – one each in black, red, green, and blue – and adds them to the shapes list.

Set the state to **HIDDEN** so that the shape doesn't appear on the screen when the program starts. It has to wait its turn.



Don't forget to save your work.

```
shapes = []

circle = c.create_oval(35, 20, 365, 350, outline='black', fill='black', state=HIDDEN)
shapes.append(circle)
circle = c.create_oval(35, 20, 365, 350, outline='red', fill='red', state=HIDDEN)
shapes.append(circle)
circle = c.create_oval(35, 20, 365, 350, outline='green', fill='green', state=HIDDEN)
shapes.append(circle)
circle = c.create_oval(35, 20, 365, 350, outline='blue', fill='blue', state=HIDDEN)
shapes.append(circle)

c.pack()
```

These are the (x0, y0) coordinates (see box).

These are the (x1, y1) coordinates (see box).

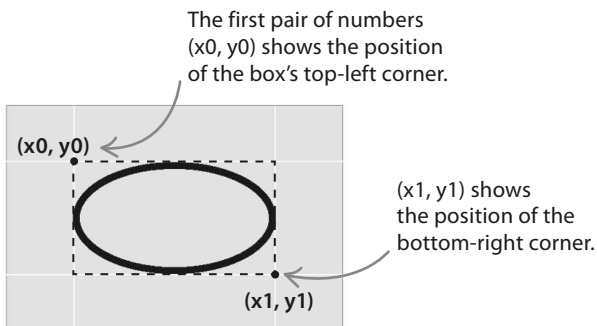
This line puts the shapes onto the canvas. Without it, none of the shapes would be displayed.

The circle's colour is determined by **outline** and **fill**.

EXPERT TIPS

Create ovals

The `create_oval()` function draws an oval as if it's inside an invisible box. The four numbers within the brackets decide the position of the circles on the screen. They are the coordinates of two opposing corners of the box. The greater the difference between the two pairs of numbers, the bigger the circle.



7 Show the circles

Try running the program. Do you see any shapes? Remember that you set their states to **HIDDEN**. Change one shape's state to **NORMAL** and run the code again. You should now be able to see that shape on the screen. Be careful not to set more than one shape to **NORMAL**. If you do, they'll all show at once, drawn one on top of the other.

